

AI Assisted Coding

B.GEETHANJALI

(2303A51005)

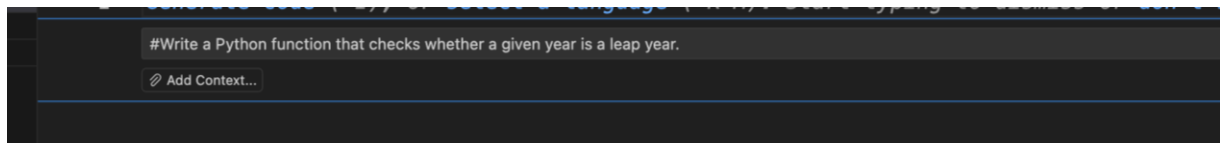
BATCH-01

Lab Experiment: Prompt Engineering – Improving Prompts and Context Management

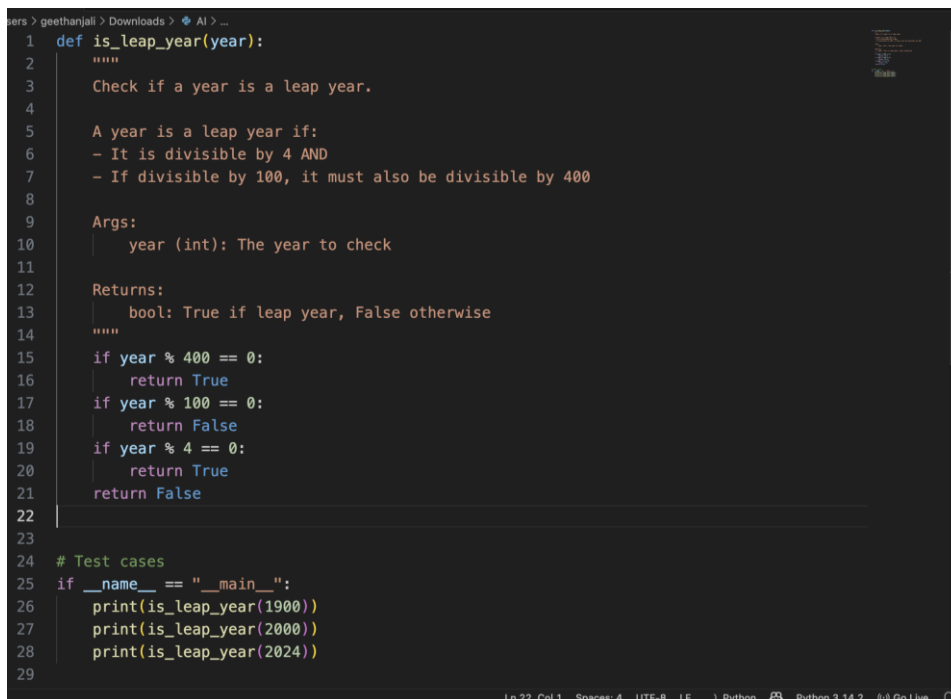
Question 1: Zero-Shot Prompting (Leap Year Check)

Write a zero-shot prompt to generate a Python function that checks whether a given year is a leap year.

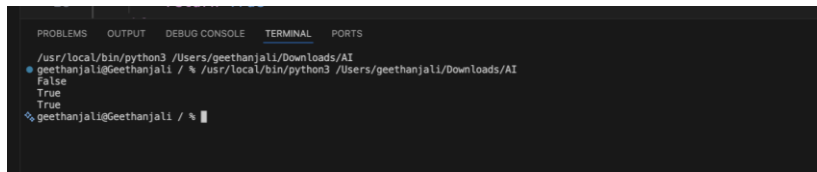
Prompt:



Copilot's suggestion:



Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
• geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
False
True
True
geethanjali@Geethanjali / %
```

Identify logical flaws:

- 1.The function does not check whether the input is an integer, so giving a string or decimal value can cause an error.
- 2.It does not handle negative years, which do not make sense for leap year calculation in real life.
- 3.The logic assumes leap year rules work for all years, even though these rules were introduced only after the Gregorian calendar.
- 4.The function allows very large or unrealistic year values without any validation.
- 5.The logic works correctly, but using many if statements makes it harder to understand and could be written in a simpler way.

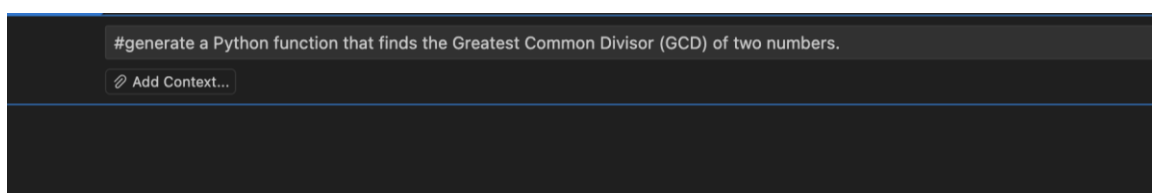
Question 2: One-Shot Prompting (GCD of Two Numbers)

Write a one-shot prompt with one example to generate a Python function that finds the Greatest Common Divisor (GCD) of two numbers.

Example:

Input: 12,18→ Output: 6

Prompt:



```
#generate a Python function that finds the Greatest Common Divisor (GCD) of two numbers.
Add Context...
```

Copilot's suggestion:

```

Users > geethanjali > Downloads > AI > ...
1  def gcd(a, b):
2      """
3      Find the Greatest Common Divisor of two numbers.
4
5      Args:
6          a: First integer
7          b: Second integer
8
9      Returns:
10         The GCD of a and b
11     """
12     a, b = abs(a), abs(b)
13     while b:
14         a, b = b, a % b
15     return a
16
17
18 # Example usage
19 if __name__ == "__main__":
20     print(gcd(12, 18))

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
6
● geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
6
◇ geethanjali@Geethanjali / % █

```

Compare the generated code with a zero-shot solution.

A zero-shot solution usually uses a simple logic where it checks all numbers from 1 up to the smaller number and finds the greatest one that divides both values. This method works, but it is slow because it checks many unnecessary numbers.

Analyze algorithm efficiency :

1. Time Complexity: $O(\log \min(a, b))$
2. Very fast, even for large numbers.
3. Widely used in mathematics and computer science.

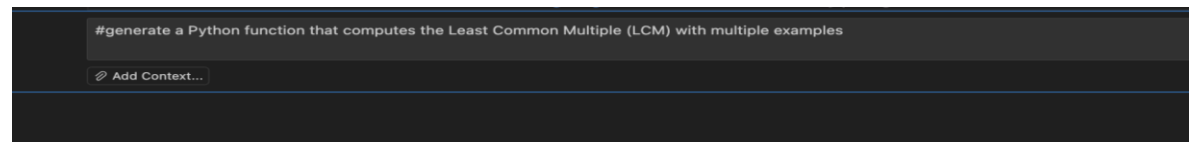
Zero-Shot Brute Force Method

1. Time Complexity: $O(\min(a, b))$
2. Slow for large inputs.
3. Not efficient for real-world applications.

Question 3: Few-Shot Prompting (LCM Calculation)

Write a few-shot prompt with multiple examples to generate a Python function that computes the Least Common Multiple (LCM).

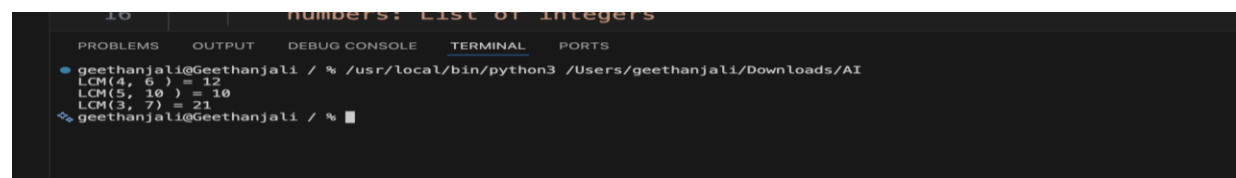
Prompt:



Copilot's suggestion:

```
Users > geethanjali > Downloads > AI > ...
1  import math
2  def lcm(a, b):
3      """
4      Compute the Least Common Multiple of two numbers.
5      Args:
6      a: First integer
7      b: Second integer
8      Returns:
9      The least common multiple of a and b
10     """
11     return abs(a * b) // math.gcd(a, b)
12 def lcm_multiple(numbers):
13     """
14     Compute the LCM of multiple numbers.
15     Args:
16     numbers: List of integers
17     Returns:
18     The least common multiple of all numbers
19     """
20     result = numbers[0]
21     for num in numbers[1:]:
22         result = lcm(result, num)
23     return result
24 # Examples
25 if __name__ == "__main__":
26     print(f"LCM(4, 6) = {lcm_multiple([4, 6])}")
27     print(f"LCM(5, 10) = {lcm_multiple([5, 10])}")
28     print(f"LCM(3, 7) = {lcm_multiple([3, 7])}")
```

Output:

A screenshot of a terminal window with a dark background. The terminal shows the execution of a Python script. The output is: LCM(4, 6) = 12, LCM(5, 10) = 10, LCM(3, 7) = 21. The prompt character is a blue fish icon. The terminal title bar shows "PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS".

```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
LCM(4, 6) = 12
LCM(5, 10) = 10
LCM(3, 7) = 21
geethanjali@Geethanjali / %
```

Influence of Multiple Examples:

Providing multiple examples helped the AI clearly understand:

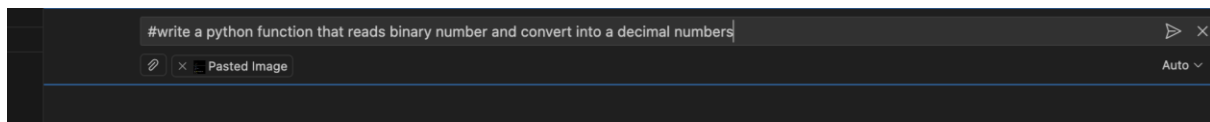
What an Armstrong number is
The expected output format
The logic required to compute digit powers

Generated code was **more structured, readable, and accurate** compared to zero shot and one-shot prompting.

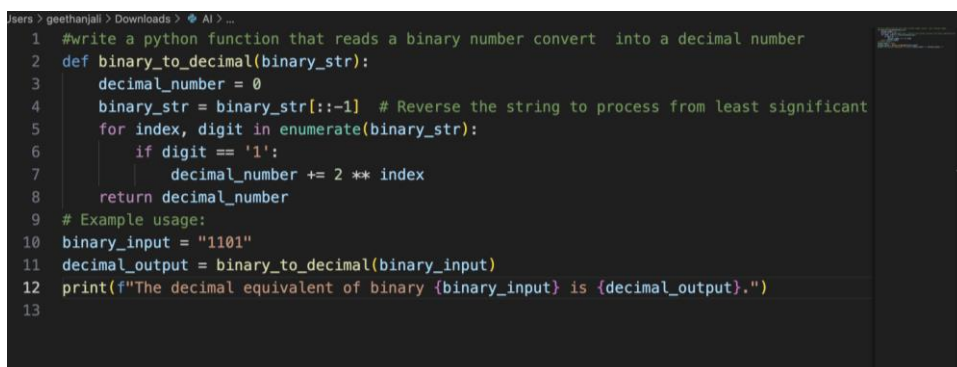
Question 4: Zero-Shot Prompting (Binary to Decimal Conversion)

Write a zero-shot prompt to generate a Python function that converts a binary number to decimal.

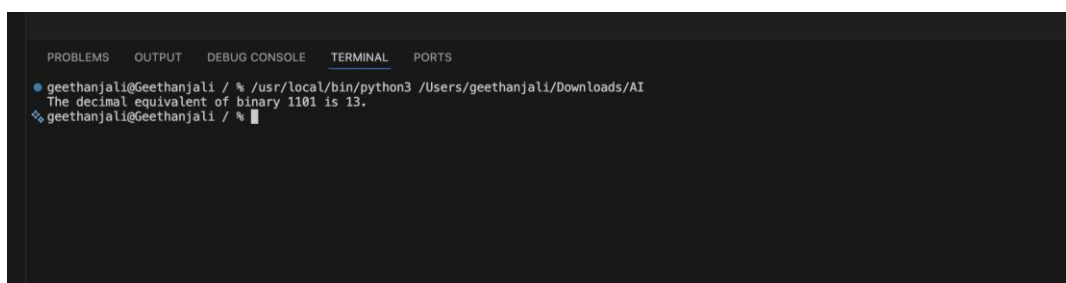
Prompt:



Copilot's suggestion:



Output:



Explanation:

1. It does not check for empty input (an empty string passes all() and is treated as valid).
2. It does not handle leading/trailing spaces (e.g., " 101 " should be stripped).
3. The try-except ValueError is unnecessary, because no ValueError can occur in this code.
4. It does not give feedback if the input is too large, which could cause performance issues.
5. The validation only checks characters, not user intent or format (like negative sign or prefixes such as 0b).

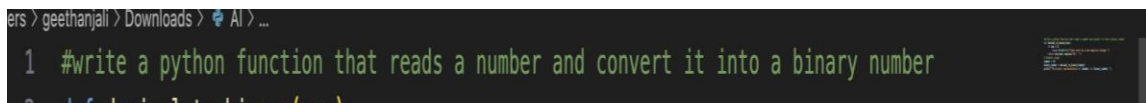
Question 5: One-Shot Prompting (Decimal to Binary Conversion)

Write a one-shot prompt with an example to generate a Python function that converts a decimal number to binary.

Example:

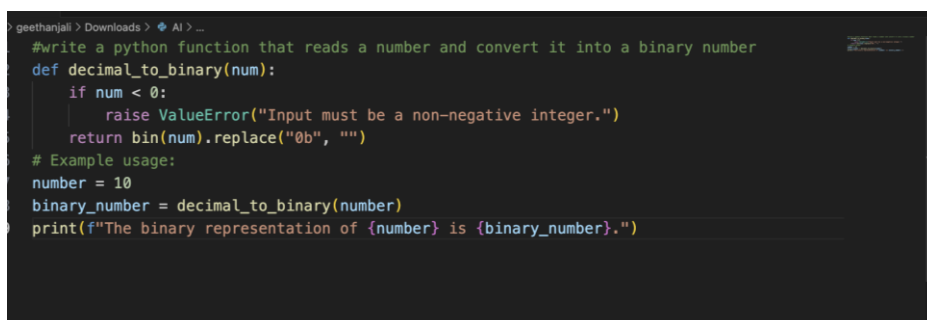
Input: 10 → Output: 1010

Prompt:



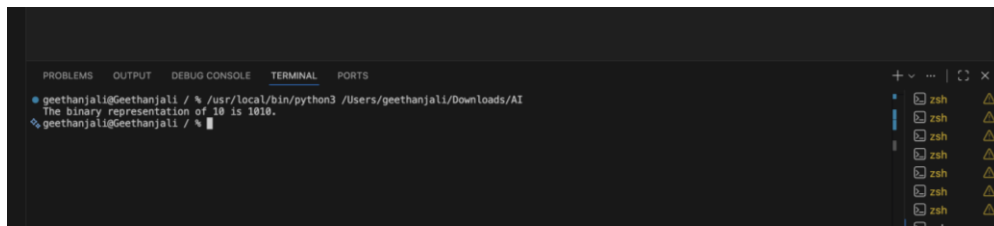
```
ers > geethanjali > Downloads > AI > ...  
1 #write a python function that reads a number and convert it into a binary number  
2 def decimal_to_binary(num):
```

Copilot's suggestion:



```
geethanjali > Downloads > AI > ...  
#write a python function that reads a number and convert it into a binary number  
def decimal_to_binary(num):  
    if num < 0:  
        raise ValueError("Input must be a non-negative integer.")  
    return bin(num).replace("0b", "")  
# Example usage:  
number = 10  
binary_number = decimal_to_binary(number)  
print(f"The binary representation of {number} is {binary_number}.")
```

Output:



Question 6: Few-Shot Prompting (Harshad Number Check)

Write a few-shot prompt to generate a Python function that checks whether a number is a Harshad (Niven) number.

Examples:

- Input: 18 → Output: Harshad Number
- Input: 21 → Output: Harshad Number
- Input: 19 → Output: Not a Harshad Number

Copilot's suggestion:

```
Users > geethanjali > Downloads > AI > ...
1  """
2  n=18
3  display Harshad Number
4  n=19
5  display Not a Harshad Number
6  n=-5
7  display Invalid Input
8  n=abc
9  display Invalid Input
10 """
11 def is_harshad_number(n):
12     try:
13         num = int(n)
14         if num <= 0:
15             return "Invalid Input"
16         digit_sum = sum(int(digit) for digit in str(num))
17         if num % digit_sum == 0:
18             return "Harshad Number"
19         else:
20             return "Not a Harshad Number"
21     except ValueError:
22         return "Invalid Input"
23 # Test cases
24 print(is_harshad_number(18)) # Harshad Number
25 print(is_harshad_number(19)) # Not a Harshad Number
26 print(is_harshad_number(-5)) # Invalid Input
```

Output:

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Harshad Number
Not a Harshad Number
Invalid Input
geethanjali@Geethanjali / %
```

Robustness of the code:

- 1.The code safely handles non-numeric input using try-except, preventing crashes.
- 2.It checks for invalid values (zero or negative numbers) before processing.
- 3.The digit sum is calculated reliably by converting the number to a string.
- 4.Returning clear messages like "Harshad Number" and "Invalid Input" improves user feedback.

Overall, the program is robust for normal user input, though returning booleans instead of strings would improve reusability.